CSC2042S Assignment 2 ~ Adam Foflonker

Preamble

3rd Party Libraries Used

- NumPy
- Pillow
- Scikit-Learn
- Matplotlib

Running the Notebook

Prerequisites for running `notebook-assignment2.ipynb`:

- Aforementioned 3rdParty libraries must be installed.
- Datasets must be placed in `"data/rgb"` and `"data/grayscale"`. For example, all training data for the rgb set must be available in `"data/rgb/train/"`.
- Note that the `"data/"` directory has been ignored by git.

Global Constants, State and Flags

A constant *random state* was defined to ensure code reproducibility accross different machines. The following hyperparameters were defined as well:

```
TEST_SIZE: float = 0.3  # 70% of dataset used for training
RANDOM_SEED: int = 42

np.random.seed(RANDOM_SEED)
random.seed(RANDOM_SEED)

EPOCHS: int = 50
PATIENCE: int = 10 # Was 3
LEARNING_RATE: float = 0.001 # Was 0.01
```

Tasks

Data Processing

The *grayscale* and *RGB* datasets were loaded into separate `train/test` arrays. Each of these arrays were then partitioned into **training**, **validation** and **testing** sets. The size of each partitioned dataset is as follows:

- Training: **70%** of original dataset
- Validation: **15%** of original dataset
- Testing: **15%** of original dataset

Multi-class Perceptron Implementation

A `BinaryPerceptron` class was implemented with the following fields:

- `weights`: An array of weights for the perceptron (cardinality equaling number of features)
- `bias`: The floating-point bias for the perceptron
- `lr`: The floating-point learning rate for the perceptron

This `BinaryPerceptron` was then used to implement the `MultiClassPerceptron` class, which contans the following fields:

- `classes`: The number of classes
- `models`: A list of all `BinaryPerceptron` instances required, where each `BinaryPerceptron` maps to a single class.

Training

A perceptron training function was implemented with the following signature:

```
def train_perceptron(model, X_train, y_train, X_val, y_val, epochs, stop_criteria,
patience) -> dict[str, list]
```

This function allows for one of two stopping criterions to be used when training, namely:

- `"epochs"`: Training ends when all epochs have been iterated.
- `"early_stop"`: Training ends when the training loop has iterated for `"patience"` number of times without any parameter adjustments being made.

Training data was shuffled each epoch to improve generalisation.

Hyperparameter Tuning

- A learning rate of `0.001` was chosen as this yielded a higher accuracy than a learning rate of `0.01`.
- Perceptron weights were **zero initialized**.
- MinMax and Z-score normalization were implemented, but Z-score normalization was chosen for the final model.
- A max epochs of `50`. Higher values such as `200` resulted in almost zero improvement over `50` epochs, which is why `50` was selected as the optimal value.
- A patience of `10` was used for *early stopping* to ensure proper convergence of perceptrons when training.

Evaluation

Evaluation metrics used:

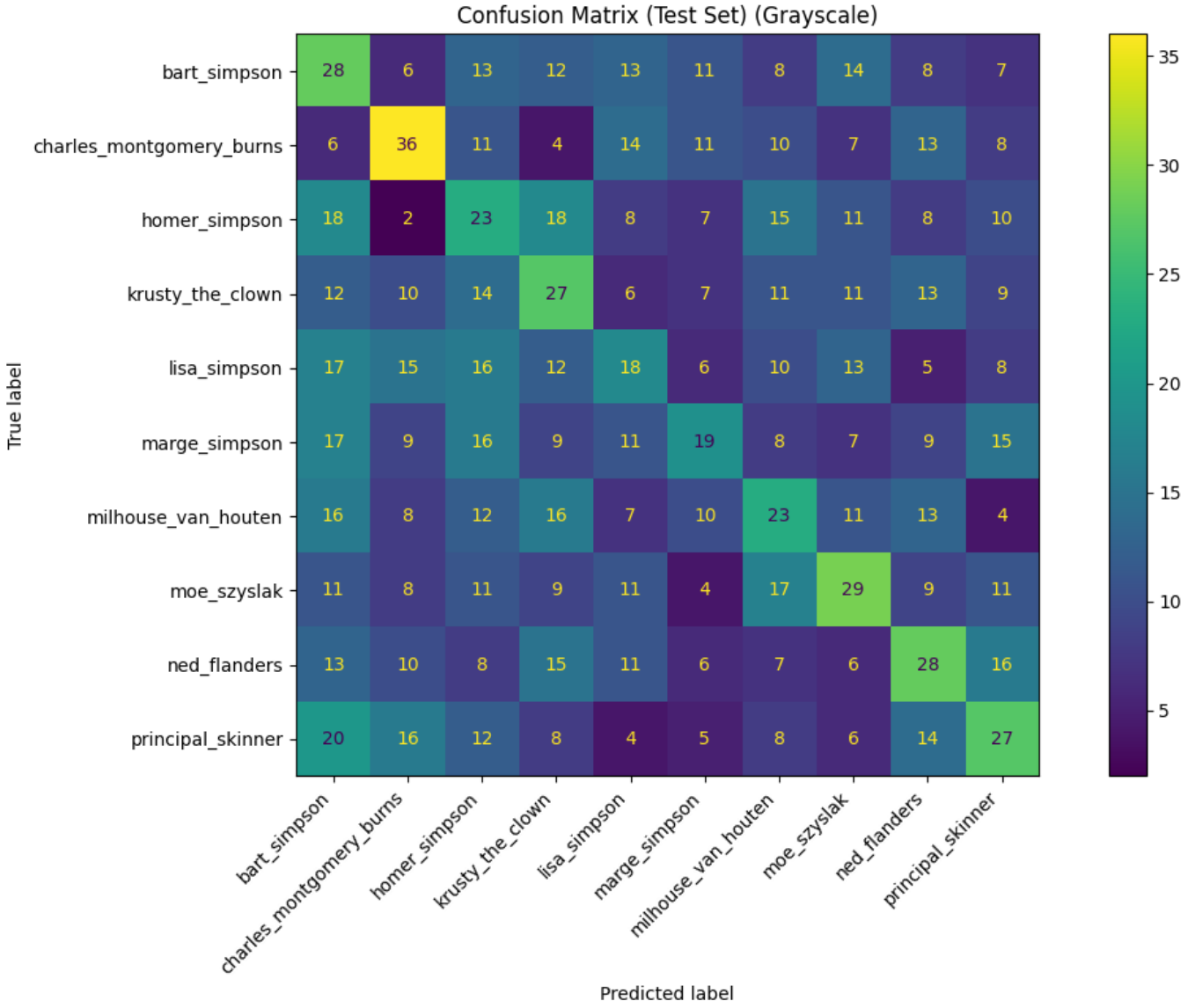
- Accuracy (acc)
- Precision (prec/P)
- Recall (rec/R)
- F1-score (f1/F1)

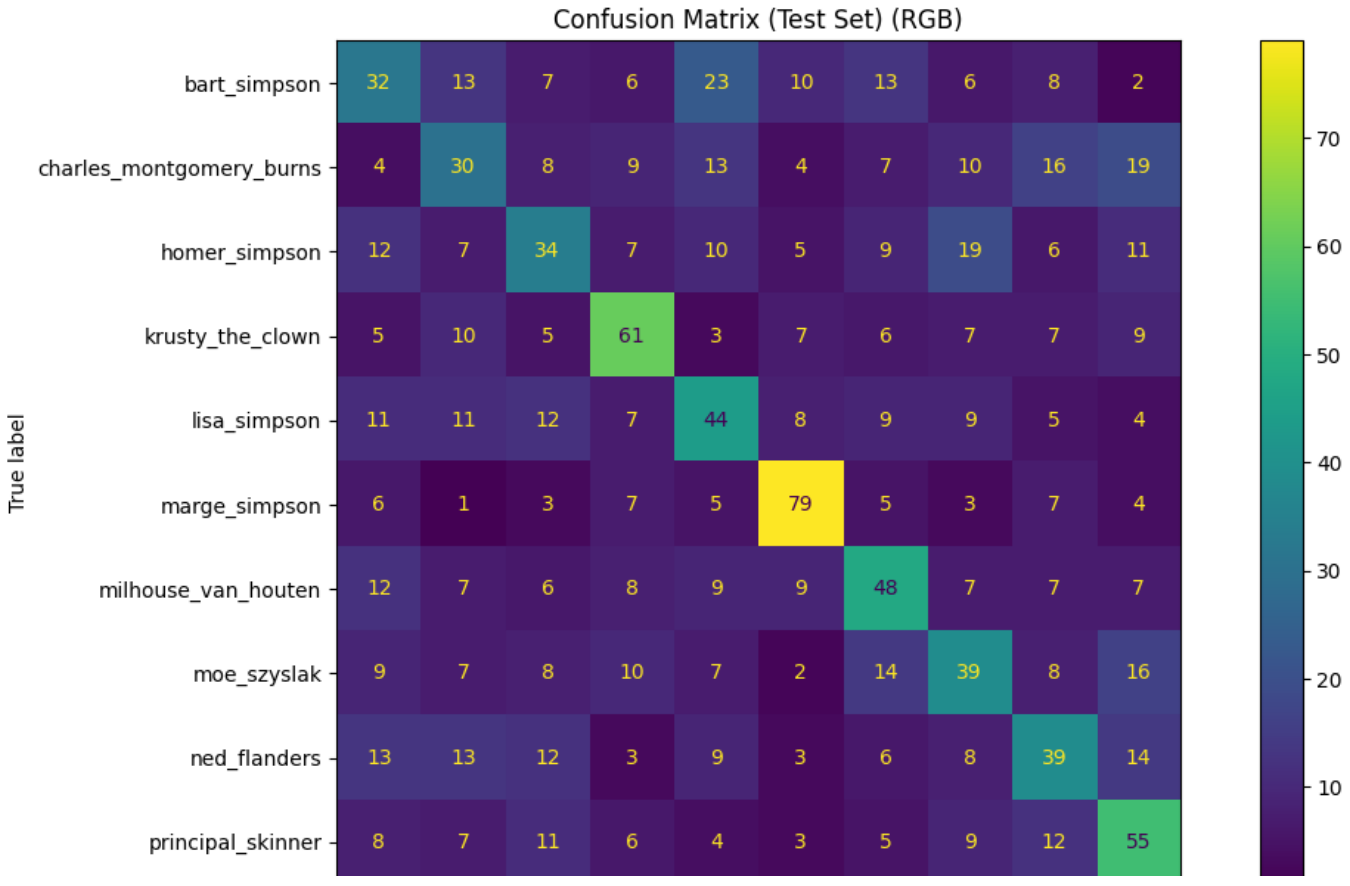
- Confusion matrix (per-class)

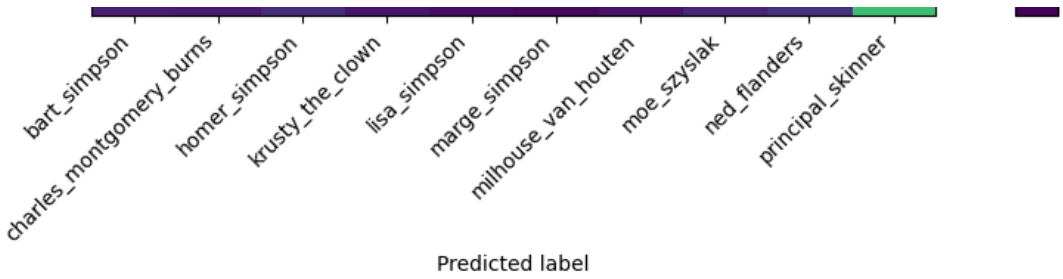
Evaluation metrics for *RGB vs Grayscale* datasets:

```
=== Overall Comparison ===
Grayscale Test: {'acc': 0.215, 'prec': 0.21665837103897467, 'rec': 0.215, 'f1':
0.21470492509352185}
RGB Test: {'acc': 0.38416666666666666, 'prec': 0.37915496681491845, 'rec':
0.38416666666666666, 'f1': 0.38096880252661763}

=== Per-class Comparison ===
bart_simpson              | Gray: P = 0.18, R = 0.23, F1 = 0.20 || RGB: P = 0.29,
R = 0.27, F1 = 0.28
charles_montgomery_burns  | Gray: P = 0.30, R = 0.30, F1 = 0.30 || RGB: P = 0.28,
R = 0.25, F1 = 0.27
homer_simpson             | Gray: P = 0.17, R = 0.19, F1 = 0.18 || RGB: P = 0.32,
R = 0.28, F1 = 0.30
krusty_the_clown          | Gray: P = 0.21, R = 0.23, F1 = 0.22 || RGB: P = 0.49,
R = 0.51, F1 = 0.50
lisa_simpson              | Gray: P = 0.17, R = 0.15, F1 = 0.16 || RGB: P = 0.35,
R = 0.37, F1 = 0.36
marge_simpson             | Gray: P = 0.22, R = 0.16, F1 = 0.18 || RGB: P = 0.61,
R = 0.66, F1 = 0.63
milhouse_van_houten       | Gray: P = 0.20, R = 0.19, F1 = 0.19 || RGB: P = 0.39,
R = 0.40, F1 = 0.40
moe_szyslak               | Gray: P = 0.25, R = 0.24, F1 = 0.25 || RGB: P = 0.33,
R = 0.33, F1 = 0.33
ned_flanders              | Gray: P = 0.23, R = 0.23, F1 = 0.23 || RGB: P = 0.34,
R = 0.33, F1 = 0.33
principal_skinner         | Gray: P = 0.23, R = 0.23, F1 = 0.23 || RGB: P = 0.39,
R = 0.46, F1 = 0.42
```


Confusion Matrix (Test Set) (Grayscale)
bart_simpson 28 6 13 12 13 11 8 14 8 7
charles_montgomery_burns 6 36 11 4 14 11 10 7 13 8
homer_simpson 18 2 23 18 8 7 15 11 8 10
krusty_the_clown 12 10 14 27 6 7 11 11 13 9
lisa_simpson 17 15 16 12 18 6 10 13 5 8
marge_simpson 17 9 16 9 11 19 8 7 9 15
milhouse_van_houten 16 8 12 16 7 10 23 11 13 4
moe_szyslak 11 8 11 9 11 4 17 29 9 11
ned_flanders 13 10 8 15 11 6 7 6 28 16
principal_skinner 20 16 12 8 4 5 8 6 14 27
True label
bart_simpson
charles_montgomery_burns
homer_simpson
krusty_the_clown
lisa_simpson
marge_simpson
milhouse_van_houten
moe_szyslak
ned_flanders
principal_skinner
Predicted label
35
30
25
20
15
10
5


Confusion Matrix (Test Set) (RGB)
bart_simpson 32 13 7 6 23 10 13 6 8 2
charles_montgomery_burns 4 30 8 9 13 4 7 10 16 19
homer_simpson 12 7 34 7 10 5 9 19 6 11
krusty_the_clown 5 10 5 61 3 7 6 7 7 9
lisa_simpson 11 11 12 7 44 8 9 9 5 4
marge_simpson 6 1 3 7 5 79 5 3 7 4
milhouse_van_houten 12 7 6 8 9 9 48 7 7 7
moe_szyslak 9 7 8 10 7 2 14 39 8 16
ned_flanders 13 13 12 3 9 3 6 8 39 14
principal_skinner 8 7 11 6 4 3 5 9 12 55
True label
70
60
50
40
30
20
10



On inspection of the above confusion matrices, it can be determined that the most accurately predicted class is `marge_simpson`, likely due to the character's big blue hair. And it appears that the least accurately predicted class is `charles_montgomery_burns`. It also appears that `bart_simpson` and `lisa_simpson` are commonly confused.

RGB vs Grayscale Analysis

The **RGB model** has a significantly higher *accuracy, precision, recall and F1-score* than the **grayscale model.** The **RGB model** also performs better per-class than the **grayscale model** however, this is not true for all classes. For instance, the **grayscale model** predicts `charles_montgomery_burns` more accurately than the **RGB model**. Although the **grayscale model** tends to confuse the classes much more than the **RGB model** due to the limited color range (neutrals only).

`marge_simpson` Benefits greatly from color cues due to her big blue hair - which is also likely the reason why she is the most accurately predicted class. While on the other hand, classes like `bart_simpson`, `homer_simpson` and `charles_montgomery_burns` get confused quite a bit even in the **RGB model**, likely due to all of them having the same *yellow skin-tone*.

Despite this challenge with the **RGB model**, it still vastly out-performs the **grayscale model**. **Grayscale** just doesn't suffice - it confuses the classes way too much therefore, **RGB** clearly provides the advantage here.

References

- ChatGPT: Debugging, pipeline overview, results/output verification.